

PROGRAMACIÓN ORIENTADA A OBJETO SEGURO

Evaluación Sumativa 4 · Unidad 4



Viajes Aventura

Informe Técnico Grupal

Desarrollo de software para una agencia de viajes: destinos, paquetes y reservas seguras

EQUIPO

Gianni Antonietti

Líder de Proyecto (Scrum Master y Product Owner)

Maximiliano Montoya

Desarrollador Backend (persistencia y seguridad)

DOCENTE

Rubén Schnettler

SECCIÓN

V-IEI-N2-P3-C1 · INACAP Valparaíso

CARRERA

Analista Programador

FECHA DE ENTREGA

05-10-2026

Índice

Índice.....	2
1. Introducción.....	4
1.1 Objetivo general.....	4
1.2 Objetivos específicos.....	4
1.3 Alcance.....	4
2. Equipo de trabajo y roles.....	5
3. Análisis de la problemática.....	6
4. Requerimientos.....	7
4.1 Criterios de clasificación.....	7
4.2 Criterios de priorización.....	7
4.3 Estructura de redacción.....	7
4.4 Requerimientos funcionales.....	7
4.5 Requerimientos no funcionales.....	9
4.6 Resumen de prioridades.....	10
5. Verificación: requerimientos frente a la problemática.....	11
6. Modelado de la solución.....	12
6.1 Diagrama de casos de uso.....	12
6.2 Diagramas de procesos BPMN.....	15
6.3 Diagrama de clases UML.....	18
7. Trazabilidad entre requerimientos, modelos y código.....	20
8. Planificación ágil.....	21
8.1 Marco de trabajo.....	21
8.2 Product Backlog.....	21
8.3 Sprint Backlog.....	22
8.4 Cronograma y entregables.....	23
9. Supuestos.....	24
10. Implementación en Python.....	25
10.1 Estructura del proyecto.....	25
10.2 Correspondencia con el diseño y principios de POO.....	25
10.3 Persistencia de datos.....	26
10.4 Operaciones CRUD.....	26
10.5 Evidencia de ejecución.....	27
11. Seguridad del sistema.....	28
11.1 Librerías utilizadas.....	28
11.2 Autenticación.....	28
11.3 Validación de credenciales y control de acceso.....	28
11.4 Protección de la información sensible.....	28
11.5 Validación de entradas y otros controles.....	29
11.6 Pruebas de seguridad.....	29
11.7 Evaluación de la seguridad con apoyo de IA.....	30
12. Uso crítico de herramientas de IA.....	32
12.1 Herramientas utilizadas.....	32
12.2 Iteraciones de propuesta del modelo.....	32
12.3 Análisis del modelo propuesto por la IA.....	32
12.4 Análisis del código generado con IA.....	32
12.5 Modelo ajustado.....	33
13. Conclusiones.....	34

Referencias.....35

1. Introducción

Viajes Aventura es una agencia de Valparaíso, formada por tres socios, que arma y vende paquetes turísticos combinando destinos que ellos mismos han recorrido. Hoy no cuenta con un sistema: el catálogo vive en una planilla compartida, las reservas se anotan en un cuaderno y los datos de los clientes quedan repartidos entre la planilla, el cuaderno y los teléfonos de los socios.

Este informe documenta el desarrollo de una solución de software en Python que reemplaza ese proceso manual. Integra la gestión de destinos, paquetes turísticos y reservas, con autenticación de usuarios, persistencia en base de datos y protección de los datos sensibles de los clientes.

1.1 Objetivo general

Desarrollar un sistema en Python, orientado a objetos y con persistencia de datos, que permita a Viajes Aventura administrar su catálogo de destinos, armar y publicar paquetes, y recibir reservas de clientes autenticados de forma segura.

1.2 Objetivos específicos

- Identificar, clasificar y priorizar los requerimientos funcionales y no funcionales a partir del caso y de las entrevistas a los socios.
- Modelar la solución mediante diagramas de casos de uso, procesos BPMN y un diagrama de clases UML trazables a los requerimientos.
- Planificar el desarrollo con un enfoque ágil (Scrum), definiendo roles, Product Backlog, Sprint Backlog y entregables.
- Implementar la solución en Python con persistencia de datos y operaciones CRUD.
- Incorporar mecanismos de acceso seguro con librerías oficiales y validar críticamente las recomendaciones de herramientas de IA.

1.3 Alcance

Dentro del alcance	Fuera del alcance
Destinos: registrar, modificar, retirar (eliminar o marcar no disponible) y listar. Paquetes: crear combinando destinos, definir fechas y cupo, calcular el precio, publicar y consultar disponibilidad. Reservas: registro y autenticación de clientes, reservar un paquete, almacenar la reserva y consultar el historial propio. Seguridad: autenticación, validación de todo dato ingresado y protección de credenciales y datos sensibles.	Pasarela de pago y verificación automática de transferencias. Facturación electrónica ante el SII. Aplicación para teléfonos. Integración con aerolíneas, hoteles u operadores. Envío de correos o mensajería al cliente. Informes de gestión, contabilidad y remuneraciones.

2. Equipo de trabajo y roles

El equipo está formado por dos integrantes y trabaja con **Scrum adaptado a un equipo pequeño**. Scrum define tres responsabilidades (Product Owner, Scrum Master y Developers); con dos personas, el Líder de Proyecto asume la coordinación y la representación del cliente, y el desarrollador backend concentra la persistencia y la seguridad. Ambos programan, para repartir la carga de desarrollo de forma equilibrada.

Integrante	Rol	Responsabilidades	Por qué esta distribución
Gianni Antonietti	Líder de Proyecto (Scrum Master y Product Owner)	Coordina al equipo y las reuniones de seguimiento. Facilita la planificación, revisión y retrospectiva de cada sprint. Interpreta el caso, las entrevistas y las reglas R1 a R17, y mantiene y prioriza el Product Backlog. Define los criterios de aceptación y valida que cada historia los cumpla. Desarrolla la interfaz de consola y los menús. Integra el informe y la entrega en el AAI.	La guía exige un Líder de Proyecto. Unir la coordinación con la representación del cliente evita que las decisiones de prioridad queden dispersas en un equipo de dos.
Maximiliano Montoya	Desarrollador Backend (persistencia y seguridad)	Implementa las clases del modelo, la base de datos SQLite y sus operaciones CRUD. Implementa los mecanismos de seguridad: hash de contraseñas, cifrado del RUT y el teléfono, control de sesiones y bloqueo por intentos. Integra los servicios externos. Revisa el código propuesto por herramientas de IA antes de integrarlo.	La persistencia y la seguridad concentran la mayor ponderación técnica de la rúbrica y requieren un responsable claro y continuo.
Ambos	Equipo de desarrollo	Programar las historias del sprint, probar su propio código, revisar el código del otro y documentar sus decisiones en el informe.	En un equipo de dos, ambos deben programar para cumplir los plazos y poder defender cualquier parte del sistema.

3. Análisis de la problemática

La información se obtuvo del caso entregado por el docente: la descripción del negocio, la forma de trabajo actual, las entrevistas a los tres socios, las cifras de la última temporada y las 17 reglas de negocio vigentes. La siguiente tabla resume los problemas detectados, cada uno con su evidencia. Cada problema recibe un código (P1 a P9) que se usa después para demostrar que los requerimientos responden a él.

ID	Problema	Evidencia en el caso
P1	Reservas duplicadas	19 reservas duplicadas al cierre de temporada. Una misma reserva escrita por dos socios con nombres distintos («Carolina Reyes» y «c. reyes»).
P2	Sobreventa del cupo	6 reservas aceptadas por sobre el cupo. Matías vendió 14 lugares en un paquete de 12; un cliente ya había comprado el pasaje.
P3	Ventas de paquetes vencidos	3 reservas sobre paquetes cuya fecha de salida ya había pasado.
P4	Precio que cambia después de vender	4 paquetes cobrados a un precio distinto del publicado. Caso de Sofía Larraín: dos totales (2.100.000 y 2.475.000) para la misma reserva.
P5	Errores de cálculo manual	Paulina suma costos a mano; un error de un cero publicó un paquete a la décima parte de su costo.
P6	Catálogo desordenado	Destinos repetidos con distinto nombre; 4 destinos que ya no se operan siguen visibles, pero no se pueden borrar porque están en paquetes vendidos.
P7	Datos sensibles expuestos	RUT y teléfono en una planilla abierta desde computadores personales y enviada por correo varias veces.
P8	Sin historial consultable	31 consultas de clientes respondidas revisando el cuaderno hoja por hoja. No se sabe cuántas personas han viajado.
P9	Sin control de acceso	Los tres socios editan la planilla sin distinción de permisos; no hay registro de quién cambia qué.

Distinción entre problemas similares

Las reservas sobre el cupo (P2) y las de fechas vencidas (P3) no son el mismo problema. La primera se resuelve comparando contra un dato que el sistema debe llevar (el cupo disponible) y la segunda, comparando contra la fecha del día. Las diferencias de precio (P4) tampoco son un error de suma: ocurren porque el costo de un destino cambia después de vendido el paquete. Por eso cada uno genera un requerimiento distinto (RF-17, RF-18 y RF-09).

4. Requerimientos

4.1 Criterios de clasificación

Los requerimientos se clasifican en dos tipos. Los **funcionales (RF)** describen qué debe hacer el sistema y se agrupan por módulo: Destinos, Paquetes, Usuarios y Reservas. Los **no funcionales (RNF)** describen cómo debe comportarse y se agrupan según las características de calidad de la norma ISO/IEC 25010: seguridad, fiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad.

El caso advierte que una regla de negocio no es un requerimiento: la regla describe cómo funciona el negocio y el requerimiento, qué debe hacer el sistema. Por eso cada regla R1 a R17 se tradujo a una o más acciones verificables del sistema, y la columna «Origen» indica de qué regla, problema o exigencia de la guía proviene cada requerimiento.

4.2 Criterios de priorización

Se usa la técnica **MoSCoW**, con criterios explícitos para que la prioridad no dependa de la opinión:

Prioridad	Criterio aplicado
Must (debe)	Resuelve un problema medido del caso (P1 a P9), lo exige explícitamente la guía o el alcance acordado, o es condición de seguridad. Sin él la solución no se puede entregar.
Should (debería)	Aporta valor importante o cubre un vacío del caso, pero el sistema funciona sin él. Se implementa si el sprint lo permite.
Could (podría)	Mejora la experiencia o previene un problema menor. Se deja para el final.
Won't (no esta vez)	Está fuera del alcance acordado con los socios: pasarela de pago, facturación electrónica, app móvil, correos e informes de gestión.

Además se consideran las **dependencias**: un requerimiento que otros necesitan va primero. Por ejemplo, RF-20 (persistencia) y RF-13 (autenticación) son prerrequisito de todas las reservas, y RF-07 (destinos del paquete) lo es de RF-08 y RF-09 (precio).

4.3 Estructura de redacción

Cada requerimiento se redacta con la forma «**El sistema debe + acción + objeto + condición**», en una sola oración verificable, y se acompaña de un **criterio de aceptación** que permite comprobar si se cumple. Se evitan términos ambiguos como «rápido» o «fácil» sin una medida.

4.4 Requerimientos funcionales

ID	Módulo	Requerimiento	Prioridad	Origen	Criterio de aceptación
RF-01	Destinos	El sistema debe permitir al administrador registrar un destino con nombre, zona, descripción, duración en días y costo base por persona, rechazando nombres ya existentes en el catálogo.	Must	R1 · P6	Un destino con nombre repetido se rechaza con un mensaje claro.
RF-02	Destinos	El sistema debe validar que el costo base sea mayor que cero y la duración sea al menos un día.	Must	R2	Un costo 0 o negativo se rechaza.
RF-03	Destinos	El sistema debe permitir al administrador modificar el nombre, la zona, la descripción, la duración o el costo base de un destino, rechazando un nombre que ya exista y registrando la fecha de actualización.	Must	Guía · R1 · P6	El cambio queda guardado; un nombre repetido se rechaza; el precio de los paquetes publicados no cambia.
RF-04	Destinos	El sistema debe eliminar un destino que no forma parte de ningún paquete; si forma parte de alguno, debe marcarlo como no disponible en lugar de eliminarlo.	Must	R8 · Guía · P6	Un destino usado en un paquete queda «no disponible» y el paquete conserva su contenido.

ID	Módulo	Requerimiento	Prioridad	Origen	Criterio de aceptación
RF-05	Destinos	El sistema debe listar los destinos disponibles con su costo base, y ofrecer solo los disponibles al armar paquetes nuevos.	Must	R8 · Guía	Un destino no disponible no aparece como opción al crear un paquete.
RF-06	Paquetes	El sistema debe permitir al administrador crear un paquete con nombre, fecha de salida, fecha de regreso, cupo máximo y margen de operación (20 % por defecto), validando que la salida sea posterior a hoy, el regreso posterior a la salida, el cupo mayor que cero y el margen no negativo.	Must	R5 · R6	Fechas invertidas, cupo 0 o margen negativo se rechazan.
RF-07	Paquetes	El sistema debe permitir asociar a un paquete entre dos y cinco destinos disponibles, sin repetir un destino dentro del mismo paquete.	Must	R3 · R4	No se puede publicar con 1 destino ni agregar un sexto o uno repetido.
RF-08	Paquetes	El sistema debe calcular el precio por persona como la suma de los costos base de los destinos incluidos más el margen definido.	Must	R6 · P5	Con destinos de 310.000 y 120.000 y margen 20 %, el precio es 516.000.
RF-09	Paquetes	El sistema debe fijar el precio del paquete al publicarlo, de modo que cambios posteriores en el costo de un destino no lo modifiquen.	Must	R7 · P4	Tras cambiar el costo de un destino, el paquete publicado mantiene su precio.
RF-10	Paquetes	El sistema debe listar los paquetes vigentes (publicados y con fecha de salida posterior a la fecha actual) mostrando nombre, fecha de salida, precio por persona y cupo disponible.	Must	R15 · Guía · P3	Un paquete con salida pasada no aparece en el listado.
RF-11	Paquetes	El sistema debe calcular el cupo disponible como el cupo máximo menos las personas de las reservas no canceladas.	Must	R14 · P2	Con cupo 12 y reservas activas por 10 personas, el cupo disponible es 2.
RF-12	Usuarios	El sistema debe permitir que un visitante se registre como cliente con nombre, RUT, correo electrónico, teléfono y contraseña, rechazando correos ya registrados.	Must	R9	Un correo repetido se rechaza sin revelar datos del otro usuario.
RF-13	Usuarios	El sistema debe autenticar a los usuarios mediante correo y contraseña antes de permitir operaciones protegidas.	Must	R11 · Guía	Credenciales incorrectas no dan acceso.
RF-14	Usuarios	El sistema debe distinguir los roles Cliente y Administrador, y permitir la gestión de destinos, paquetes y pagos solo al administrador.	Must	Vacío del caso · P9	Un cliente no ve ni puede ejecutar opciones de administración.
RF-15	Usuarios	El sistema debe permitir al usuario cerrar su sesión.	Should	R11	Tras cerrar sesión, reservar exige autenticarse de nuevo.
RF-16	Reservas	El sistema debe permitir a un cliente autenticado reservar un paquete vigente indicando una cantidad de personas mayor o igual a uno.	Must	R11 · R16	Una reserva con 0 personas se rechaza.
RF-17	Reservas	El sistema debe rechazar una reserva cuya cantidad de personas supere el cupo disponible del paquete.	Must	R14 · P2	Con cupo disponible 2, una reserva de 3 personas se rechaza.
RF-18	Reservas	El sistema debe rechazar una reserva sobre un paquete cuya fecha de salida ya pasó.	Must	R15 · P3	Se rechaza aunque el paquete tenga cupo.
RF-19	Reservas	El sistema debe registrar en cada reserva la fecha de emisión, la cantidad de personas, el estado (inicia PENDIENTE) y el total, calculado como precio por persona × personas, sin permitir que el total cambie después.	Must	R12 · R13 · P4	El total de una reserva no varía aunque cambien precios o costos.
RF-20	Reservas	El sistema debe almacenar clientes, destinos, paquetes y reservas en una base de datos persistente.	Must	Guía · P1 · P8	Al cerrar y abrir el programa, los datos siguen disponibles.
RF-21	Reservas	El sistema debe permitir a un cliente autenticado consultar el historial de sus propias reservas, y solo las suyas.	Must	R11 · P8	Un cliente no puede ver reservas de otro.

ID	Módulo	Requerimiento	Prioridad	Origen	Criterio de aceptación
RF-22	Reservas	El sistema debe permitir a un cliente cancelar, previa confirmación, una reserva propia pendiente cuyo paquete aún no haya salido; la reserva pasa a cancelada y libera su cupo.	Should	Vacío del caso · R14	El cupo disponible aumenta tras la cancelación; una reserva pagada no aparece como cancelable.
RF-23	Reservas	El sistema debe permitir al administrador marcar una reserva como PAGADA después de verificar la transferencia.	Should	Forma de pago · Supuesto	El estado cambia a PAGADA y queda registrado.
RF-24	Reservas	El sistema debe advertir cuando un cliente intenta reservar un paquete en el que ya tiene una reserva activa.	Could	P1	Se muestra una advertencia y se pide confirmación.
RF-25	Servicios externos	El sistema debe permitir a un usuario autenticado consultar el clima actual de un destino mediante un servicio externo.	Could	Valor agregado	Se muestra estado del tiempo, temperatura y humedad, o un aviso si el servicio falla.
RF-26	Servicios externos	El sistema debe permitir a un cliente autenticado ver el precio de un paquete convertido a dólares o euros según el valor del Banco Central.	Could	Valor agregado	Se muestra el monto convertido y la fecha del tipo de cambio.
RF-27	Reservas	El sistema debe permitir al administrador listar todas las reservas, mostrando el RUT y el teléfono del cliente solo enmascarados.	Should	R17 · P8	El listado muestra *****8-5 y no el RUT completo.
RF-28	Paquetes	El sistema debe permitir al administrador completar un paquete en borrador agregando destinos y publicarlo; un paquete publicado no se modifica.	Should	R3 · R7	Un borrador con dos destinos se publica; un paquete publicado no aparece para completar.
RF-29	Paquetes	El sistema debe permitir al administrador eliminar, previa confirmación, un paquete que no tenga reservas.	Should	Guía · R12	Un paquete con reservas no se elimina; uno sin reservas solo se elimina al confirmar.

4.5 Requerimientos no funcionales

ID	Categoría	Requerimiento	Prioridad	Origen	Criterio de aceptación
RNF-01	Seguridad	Las contraseñas deben almacenarse solo como hash con sal aleatoria, generado con PBKDF2-HMAC-SHA256 (hashlib) y 600.000 iteraciones, según la recomendación vigente de OWASP, y compararse en tiempo constante; nunca en texto plano.	Must	R10 · I.19	En la base de datos no existe ninguna contraseña legible.
RNF-02	Seguridad	El RUT y el teléfono deben almacenarse cifrados con cifrado simétrico autenticado (librería cryptography, Fernet), y la clave no debe estar escrita en el código fuente.	Must	R17 · P7 · I.19	Al abrir la base de datos con un visor, el RUT y el teléfono no son legibles.
RNF-03	Seguridad	El RUT y el teléfono no deben mostrarse en listados ni en mensajes de error; cuando sea necesario, se muestran enmascarados.	Must	R17 · P7	Un listado muestra 12.*.*-5 y no el RUT completo.
RNF-04	Seguridad	La contraseña debe ingresarse sin mostrarse en pantalla.	Must	Buenas prácticas	Al escribirla no aparece el texto.
RNF-05	Seguridad	Ante un inicio de sesión fallido, el mensaje debe ser genérico («Correo o contraseña incorrectos»), y tras 3 intentos fallidos con un mismo correo el acceso debe bloquearse durante 1 minuto, exista o no ese correo.	Should	Guía · G.18	El mensaje no indica cuál dato falló; el cuarto intento se bloquea.
RNF-06	Seguridad	Las contraseñas de registro deben tener entre 8 y 64 caracteres, combinar letras y números, y confirmarse escribiéndolas dos veces.	Should	G.18	«aaaaaaa» y «12345678» se rechazan; si la confirmación no coincide, se vuelve a pedir.
RNF-07	Seguridad	Toda consulta a la base de datos debe usar parámetros, nunca concatenación de texto, para prevenir inyección SQL.	Must	OWASP	Ingresar «' OR 1=1 --» como correo no da acceso.

ID	Categoría	Requerimiento	Prioridad	Origen	Criterio de aceptación
RNF-08	Seguridad	Todo dato ingresado debe validarse en tipo, rango y formato: RUT exactamente como 12345678-9 y con dígito verificador correcto (módulo 11), correo con formato válido, celular chileno sin espacios, nombres de personas y destinos sin dígitos, fechas válidas y números dentro de sus límites.	Must	Alcance · Guía	«274894903» y «27489490-4» se rechazan como RUT.
RNF-09	Fiabilidad	La base de datos debe asegurar la integridad: claves foráneas activas, unicidad del correo y del nombre de destino sin distinguir mayúsculas, y cada escritura dentro de una transacción; si una escritura falla, el cambio también se revierte en memoria.	Must	P1 · P2	No se puede insertar una reserva sobre un paquete inexistente ni un destino con nombre repetido.
RNF-10	Fiabilidad	El programa no debe terminar abruptamente ante entradas inválidas o errores de base de datos; debe informar y permitir continuar.	Must	Calidad	Ingresar letras donde se pide un número muestra un mensaje y vuelve al menú.
RNF-11	Persistencia	Los datos deben conservarse entre ejecuciones del programa.	Must	Guía	Los datos registrados siguen disponibles al reiniciar.
RNF-12	Usabilidad	La interfaz de consola debe presentar menús numerados según el rol, mensajes claros en español y confirmación de cada operación.	Should	Calidad	Cada acción termina con un mensaje de éxito o de error.
RNF-13	Eficiencia	Cada operación debe responder en menos de 2 segundos con el volumen de una temporada (18 destinos, 12 paquetes y 300 reservas).	Could	Cifras del caso	Las consultas responden en forma inmediata con datos de prueba.
RNF-14	Mantenibilidad	El código debe seguir el diseño UML, con atributos privados y separación en módulos (modelo, persistencia, seguridad e interfaz), y analizarse con una herramienta de calidad estática (SonarCloud).	Should	G.13	Cada clase del diagrama tiene su equivalente en el código.
RNF-15	Portabilidad	El sistema debe ejecutarse en Python 3 (probado en 3.14), con sus dependencias declaradas en un archivo requirements.txt e instalables desde PyPI.	Should	Guía	Se instala con pip install -r requirements.txt.
RNF-16	Seguridad	Las sesiones deben expirar a los 30 minutos y cada operación protegida debe verificar que la sesión siga vigente y pertenezca al usuario que la usa.	Should	R11	Con una sesión vencida, reservar devuelve al menú principal.
RNF-17	Fiabilidad	Las consultas a servicios externos deben tener un tiempo máximo de espera de 10 segundos y, si fallan, mostrar un aviso sin interrumpir el programa.	Should	Calidad	Sin conexión a internet, el resto del sistema sigue funcionando.

4.6 Resumen de prioridades

Tipo	Must	Should	Could	Total
Funcionales	20	6	3	29
No funcionales	9	7	1	17

5. Verificación: requerimientos frente a la problemática

Esta matriz demuestra que cada problema detectado en el caso tiene al menos un requerimiento que lo resuelve, y explica cómo. Ningún problema queda sin cubrir.

Problema	Requerimientos	Cómo lo resuelve el sistema
P1 Reservas duplicadas	RF-20, RF-24, RNF-09	Existe un único registro central de reservas, ligado a un cliente identificado por correo único. Ya no hay versiones de la misma reserva escritas por distintos socios, y el sistema advierte si un cliente reserva dos veces el mismo paquete.
P2 Sobreventa del cupo	RF-11, RF-17, RNF-09	El cupo disponible se calcula a partir de las reservas registradas y se verifica antes de aceptar cada reserva, dentro de una transacción.
P3 Paquetes vencidos	RF-10, RF-18	Los paquetes con salida pasada no se listan, y una reserva sobre ellos se rechaza comparando con la fecha del día.
P4 Precio cambiante	RF-09, RF-19	El precio se fija al publicar y el total de la reserva se fija al reservar; un cambio de costo posterior no los altera.
P5 Errores de cálculo	RF-08	El precio lo calcula el sistema a partir de los costos registrados; ya no se copia ni se suma a mano.
P6 Catálogo desordenado	RF-01, RF-03, RF-04, RF-05	El nombre de destino es único, la fecha de actualización del costo queda registrada y los destinos que ya no se operan se marcan no disponibles sin romper los paquetes vendidos.
P7 Datos sensibles expuestos	RNF-01, RNF-02, RNF-03, RNF-07	Las contraseñas se guardan con hash, el RUT y el teléfono se cifran, nunca se muestran completos y la base de datos se consulta de forma segura.
P8 Sin historial	RF-20, RF-21	Cada cliente consulta su historial en el sistema, y los socios pueden contar reservas y viajeros con certeza.
P9 Sin control de acceso	RF-13, RF-14, RNF-05	Solo usuarios autenticados operan, y solo el administrador gestiona el catálogo y los pagos.

6. Modelado de la solución

6.1 Diagrama de casos de uso

El diagrama representa la interacción entre los usuarios y el sistema. Se identifican tres actores:

- **Visitante:** persona que aún no inicia sesión. Puede ver paquetes, consultar disponibilidad y registrarse.
- **Ciente:** visitante registrado y autenticado. Hereda lo que puede hacer el visitante (generalización) y además reserva, consulta su historial y cancela reservas.
- **Administrador:** uno de los socios. Gestiona destinos, paquetes y pagos.

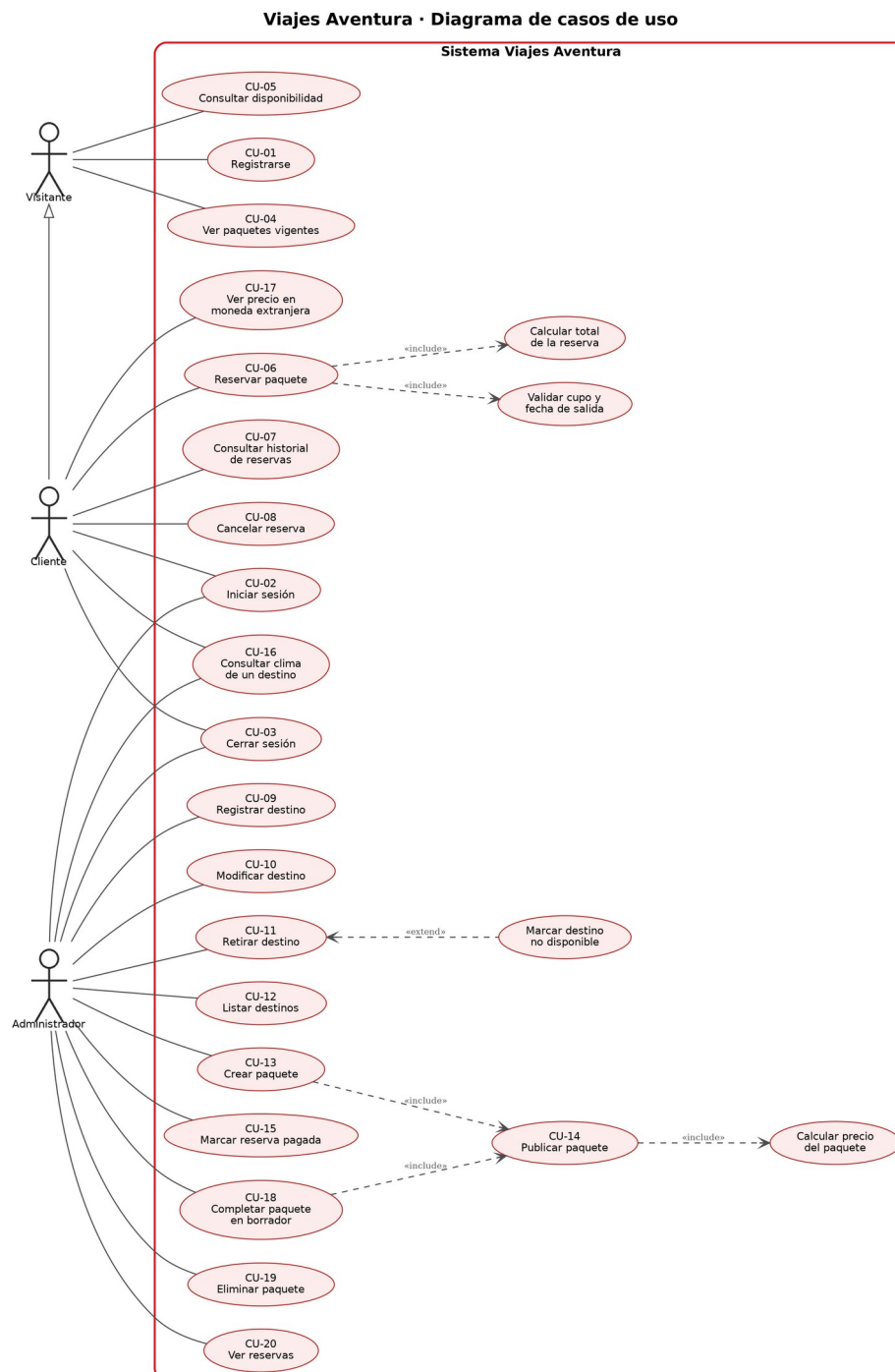


Figura 1. Diagrama de casos de uso del sistema Viajes Aventura

Las relaciones «include» indican pasos que siempre forman parte del caso de uso: toda reserva incluye validar cupo y fecha y calcular el total, al terminar de crear o completar un paquete con dos o más destinos se publica, y toda publicación incluye calcular el precio. La relación «extend» indica un comportamiento condicional: al retirar un destino, solo si forma parte de algún paquete se marca como no disponible en lugar de eliminarse (R8).

ID	Caso de uso	Actor	Requerimientos
CU-01	Registrarse	Visitante	RF-12, RNF-01, 02, 06, 08
CU-02	Iniciar sesión	Cliente, Administrador	RF-13, RF-14, RNF-04, 05
CU-03	Cerrar sesión	Cliente, Administrador	RF-15
CU-04	Ver paquetes vigentes	Visitante	RF-10
CU-05	Consultar disponibilidad	Visitante	RF-11
CU-06	Reservar paquete	Cliente	RF-16 a RF-19, RF-24
CU-07	Consultar historial de reservas	Cliente	RF-21
CU-08	Cancelar reserva	Cliente	RF-22
CU-09	Registrar destino	Administrador	RF-01, RF-02
CU-10	Modificar destino	Administrador	RF-03
CU-11	Retirar destino	Administrador	RF-04
CU-12	Listar destinos	Administrador	RF-05
CU-13	Crear paquete	Administrador	RF-06, RF-07
CU-14	Publicar paquete (se incluye al terminar CU-13)	Administrador	RF-08, RF-09
CU-15	Marcar reserva pagada	Administrador	RF-23
CU-16	Consultar clima de un destino	Cliente, Administrador	RF-25
CU-17	Ver precio en moneda extranjera	Cliente	RF-26
CU-18	Completar paquete en borrador	Administrador	RF-28
CU-19	Eliminar paquete	Administrador	RF-29
CU-20	Ver reservas	Administrador	RF-27

Especificación del caso de uso principal: CU-06 Reservar paquete

Campo	Descripción
Actor	Cliente
Precondición	El cliente inició sesión y existe al menos un paquete publicado y vigente.
Flujo principal	1. El cliente elige «Reservar». 2. El sistema verifica que la sesión siga vigente. 3. El sistema muestra los paquetes vigentes con su precio y cupo disponible. 4. El cliente selecciona un paquete e ingresa la cantidad de personas. 5. El sistema valida que la cantidad esté entre 1 y 100, que el paquete siga vigente y que haya cupo suficiente. 6. El sistema calcula el total (precio × personas), registra la reserva con estado pendiente y muestra el total.
Flujos alternativos	2a. Sesión vencida: se informa y se vuelve al menú principal. 4a. El cliente presiona Esc o elige 0: se cancela sin registrar nada. 5a. Cantidad fuera de rango: se vuelve a pedir. 5b. Paquete no vigente: se rechaza la reserva. 5c. Cupo insuficiente: se informa y se rechaza. 6a. Falla la base de datos: se revierte la reserva en memoria y se muestra un mensaje genérico.
Postcondición	La reserva queda guardada en la base de datos, el cupo disponible del paquete disminuye y el total queda fijo.

Campo	Descripción
Reglas	R11, R12, R13, R14, R15, R16

6.2 Diagramas de procesos BPMN

Los procesos de negocio se modelan con BPMN 2.0, usando carriles (lanes) para separar a los participantes: evento de inicio (círculo verde), tareas (rectángulos redondeados), compuertas exclusivas (rombo con X, donde solo se sigue un camino), eventos de fin (círculo rojo grueso) y almacén de datos (BD).

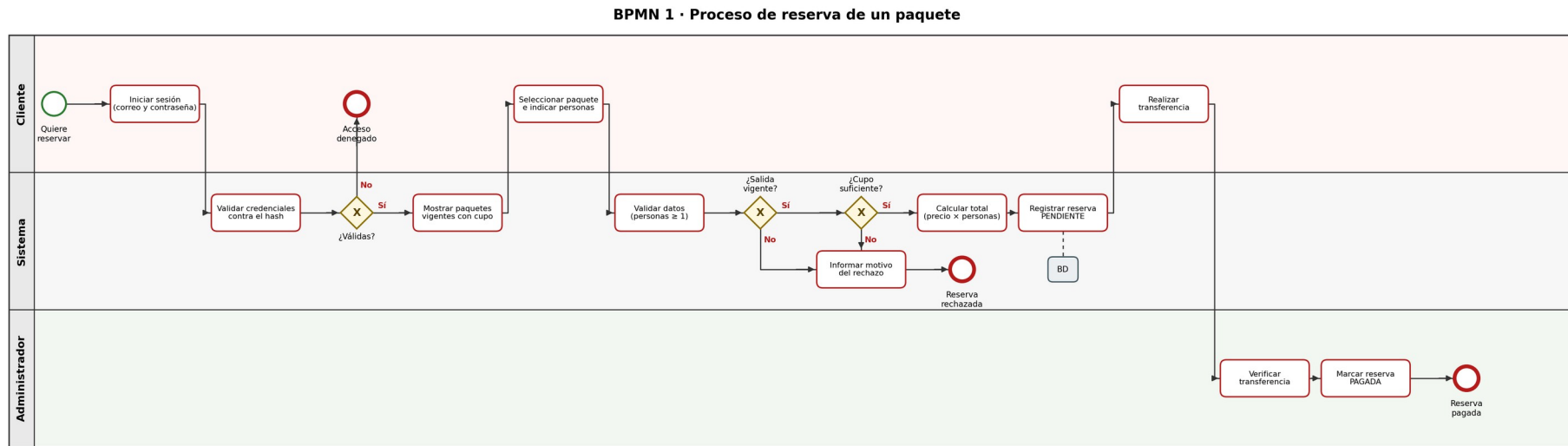


Figura 2. BPMN 1 · Proceso de reserva de un paquete

Lectura: el cliente inicia sesión y el sistema valida sus credenciales. Con acceso válido, el cliente elige un paquete y la cantidad de personas. El sistema evalúa dos condiciones distintas (fecha vigente y cupo suficiente, P3 y P2); si ambas se cumplen calcula el total, registra la reserva como PENDIENTE y el cliente transfiere. Finalmente el administrador verifica la transferencia y marca la reserva como PAGADA.

BPMN 2 · Creación y publicación de un paquete

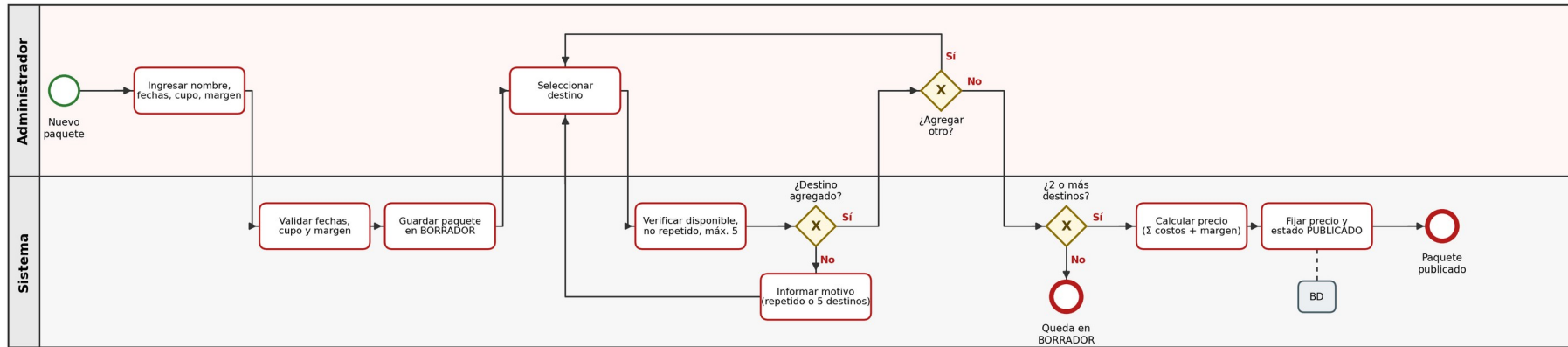


Figura 3. BPMN 2 · Creación y publicación de un paquete

Lectura: el administrador ingresa los datos del paquete y el sistema valida cada uno (R5, R6) y lo guarda como borrador. Luego el administrador agrega destinos uno a uno; el sistema rechaza los repetidos o un sexto destino y solo ofrece los disponibles (R3, R8). Al terminar, si el paquete tiene dos o más destinos, el sistema calcula y fija el precio y lo publica (R6, R7); si no, queda como borrador, que el administrador puede completar después con la opción «Completar paquete en borrador».

BPMN 3 · Registro de un cliente

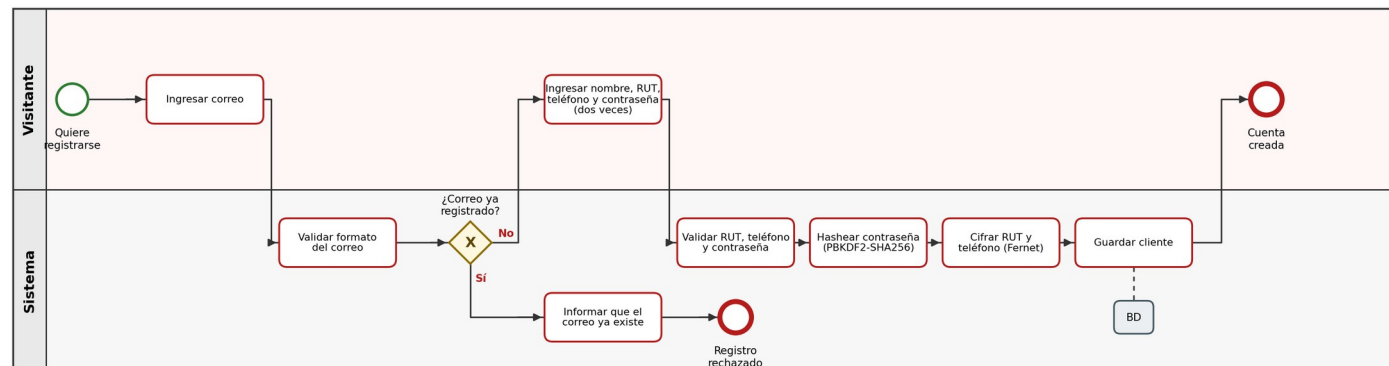


Figura 4. BPMN 3 · Registro de un cliente

Lectura: el visitante ingresa su correo y el sistema valida su formato y que no esté registrado (R9). Luego ingresa nombre, RUT, teléfono y contraseña; cada dato inválido se vuelve a pedir sin mostrar datos sensibles (R17). Con todo válido, el sistema hashea la contraseña (R10), cifra el RUT y el teléfono y guarda el cliente.

6.3 Diagrama de clases UML

El diagrama de clases sigue la notación UML 2.5.1 vista en clases: caja de tres pisos, atributos privados (-) con su tipo, métodos públicos (+) con firma completa, las cuatro relaciones (asociación, agregación, composición y generalización) y multiplicidad en ambos extremos. Esta versión se contrastó clase por clase con el código final de main.py, incluidos los métodos que se agregaron durante la implementación, por lo que diagrama y código dicen lo mismo.

Viajes Aventura · Diagrama de clases UML (modelo de dominio, main.py)

T13V21 Programación Orientada a Objeto Seguro · - privado · + público · atributos privados, métodos públicos

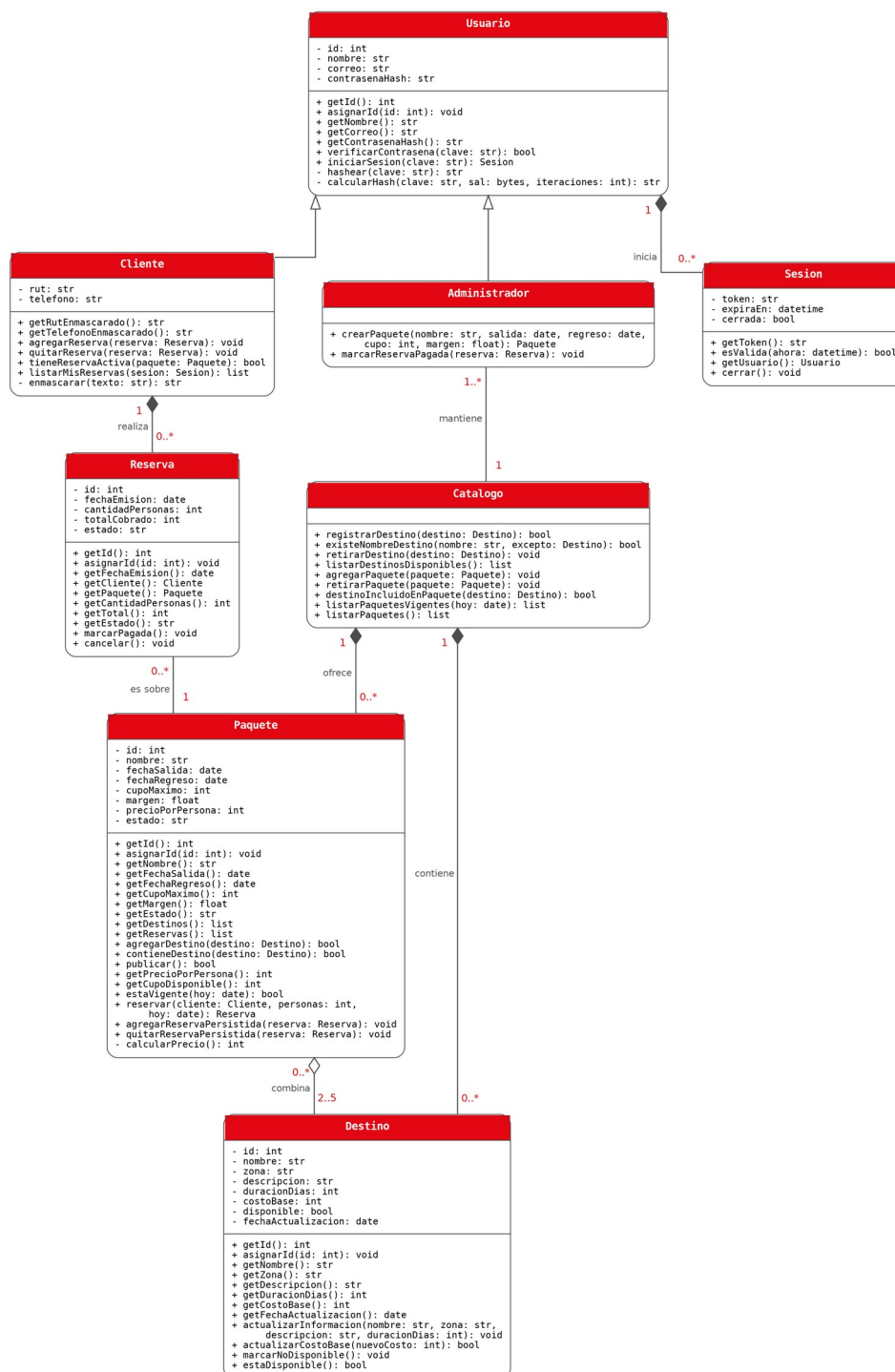


Figura 5. Diagrama de clases UML del modelo de dominio (main.py)

Relación	Tipo	Justificación
Cliente y Administrador → Usuario	Generalización	«Un cliente es un usuario» y «un administrador es un usuario». El hash, la verificación de contraseña y el inicio de sesión se escriben una sola vez.
Usuario ◆— Sesión (1 a 0..*)	Composición	Una sesión no existe sin su usuario.
Cliente ◆— Reserva (1 a 0..*)	Composición	Una reserva no tiene sentido sin el cliente que la hizo (R12). Un cliente nuevo puede no tener reservas.
Reserva — Paquete (0..* a 1)	Asociación	El paquete existe aunque nadie lo reserve; cada reserva es sobre un solo paquete.
Catálogo ◆— Destino y Paquete (1 a 0..*)	Composición	Un destino eliminado del catálogo deja de existir (R8).
Paquete ◇— Destino (0..* a 2..5)	Agregación	El destino sobrevive si se elimina el paquete y puede estar en varios a la vez (R3, R4).
Administrador — Catálogo (1..* a 1)	Asociación	Los socios mantienen el catálogo, pero ninguno es parte del otro.

Clases de soporte

Además del modelo de dominio, el sistema tiene clases técnicas: la persistencia (BaseDatos), el cifrado (Cifrador) y los servicios externos (ServicioClima y ServicioCambio). Se presentan en un diagrama aparte para que el modelo del negocio se lea con claridad. BaseDatos agrega un Cifrador porque lo recibe ya construido y lo usa para proteger el RUT y el teléfono antes de guardarlos.

Viajes Aventura · Clases de soporte (database.py, cifrado.py, servicios_externos.py)
T13V21 Programación Orientada a Objeto Seguro · - privado · + público · atributos privados, métodos públicos



Figura 6. Clases de soporte (database.py, cifrado.py y servicios_externos.py)

La interfaz de consola (interface.py) no es una clase: es un módulo de funciones que muestra los menús, valida las entradas y coordina las clases del modelo con la base de datos. En el código Python los métodos de BaseDatos se escriben en snake_case (crear_usuario) y en el diagrama en camelCase (crearUsuario), según la convención vista en clases.

7. Trazabilidad entre requerimientos, modelos y código

La matriz conecta cada requerimiento funcional con el caso de uso que lo representa, el proceso BPMN donde aparece, la clase y el método que lo implementan, y su estado en el código entregado. Así se verifica que el diseño cubre todos los requerimientos y que no hay elementos del modelo sin un requerimiento que los justifique.

Req.	Caso de uso	BPMN	Clase · método	Estado
RF-01, 02	CU-09	—	Catalogo.registrarDestino() · constructor de Destino	Implementado
RF-03	CU-10	—	Destino.actualizarInformacion() · actualizarCostoBase() · Catalogo.existeNombreDestino()	Implementado
RF-04	CU-11 «extend»	—	Catalogo.retirarDestino() · Destino.marcarNoDisponible()	Implementado
RF-05	CU-12	—	Catalogo.listarDestinosDisponibles()	Implementado
RF-06	CU-13	BPMN 2	Administrador.crearPaquete() · constructor de Paquete	Implementado
RF-07	CU-13	BPMN 2	Paquete.agregarDestino() · agregación 2..5	Implementado
RF-08	CU-14 «include»	BPMN 2	Paquete.calcularPrecio() (privado)	Implementado
RF-09	CU-14	BPMN 2	Paquete.publicar() · precioPorPersona	Implementado
RF-10	CU-04	BPMN 1	Catalogo.listarPaquetesVigentes() · Paquete.estaVigente()	Implementado
RF-11	CU-05	BPMN 1	Paquete.getCupoDisponible()	Implementado
RF-12	CU-01	BPMN 3	Cliente · Usuario.hashear() · BaseDatos.crearUsuario()	Implementado
RF-13	CU-02	BPMN 1	Usuario.iniciarSesion() · verificarContraseña()	Implementado
RF-14	CU-02	—	Generalización Usuario → Cliente / Administrador	Implementado
RF-15	CU-03	—	Sesion.cerrar()	Implementado
RF-16 a 18	CU-06 «include»	BPMN 1	Paquete.reservar() · estaVigente() · getCupoDisponible()	Implementado
RF-19	CU-06 «include»	BPMN 1	Reserva · totalCobrado sin método para cambiarlo	Implementado
RF-20	Todos	BPMN 1, 2, 3 (BD)	BaseDatos (SQLite)	Implementado
RF-21	CU-07	—	Cliente.listarMisReservas(sesion)	Implementado
RF-22	CU-08	—	Reserva.cancelar() · Paquete.getCupoDisponible()	Implementado
RF-23	CU-15	BPMN 1	Administrador.marcarReservaPagada() · Reserva.marcarPagada()	Implementado
RF-24	CU-06	—	Cliente.tieneReservaActiva()	Implementado
RF-25	CU-16	—	ServicioClima.buscarUbicacion() · consultarClima()	Implementado
RF-26	CU-17	—	ServicioCambio.convertir()	Implementado
RF-27	CU-20	—	Paquete.getReservas() · Cliente.getRutEnmascarado()	Implementado
RF-28	CU-18 «include»	BPMN 2	Catalogo.listarPaquetes() · Paquete.agregarDestino() · publicar()	Implementado
RF-29	CU-19	—	Paquete.getReservas() · Catalogo.retirarPaquete()	Implementado

Resumen: los 29 requerimientos funcionales están implementados, incluidos todos los de prioridad Must, Should y Could.

8. Planificación ágil

8.1 Marco de trabajo

El equipo aplica **Scrum** con sprints de una semana, ajustados a las tres semanas de trabajo de la evaluación (semanas 15 a 17 de la asignatura), con la entrega en la semana 18. Se eligió un sprint semanal porque permite revisar el avance con el docente cada semana y corregir a tiempo.

El código se construyó de forma incremental desde las unidades anteriores: en la Unidad 2 se implementó el modelo UML, los principios de POO, la persistencia y el manejo de errores, y en la Unidad 3 los servicios externos y los mecanismos básicos de seguridad. Los sprints de la Unidad 4 completan el análisis y el modelado formales, alinean el código con el diseño final, cierran las historias pendientes y documentan la solución.

Evento	Cuándo	Para qué
Planificación del sprint	Inicio de cada semana (30 min)	Elegir historias del Product Backlog y dividir las en tareas con responsable.
Seguimiento diario	Mensaje breve en el grupo del equipo, lunes a viernes	Qué hice, qué haré, qué me bloquea.
Revisión del sprint	Fin de cada semana (30 min)	Mostrar lo terminado y aceptarlo con los criterios de aceptación.
Retrospectiva	Después de la revisión (15 min)	Qué mejorar en el sprint siguiente.

Definición de terminado: una historia está terminada cuando cumple su criterio de aceptación, se probó con datos válidos e inválidos, no expone datos sensibles, el código está en el repositorio y, si se usó IA, la decisión quedó registrada para el informe.

8.2 Product Backlog

Las historias de usuario se ordenan por prioridad MoSCoW y por dependencia. La estimación usa puntos de historia (escala de Fibonacci: 1, 2, 3, 5, 8), que miden esfuerzo relativo.

ID	Historia de usuario	Requerimientos	Prioridad	Puntos	Sprint	Estado
HU-01	Como administrador quiero registrar destinos para armar el catálogo.	RF-01, 02	Must	3	2	Terminada
HU-02	Como administrador quiero modificar los datos de un destino para mantenerlos al día.	RF-03	Must	2	2	Terminada
HU-03	Como administrador quiero retirar un destino sin romper los paquetes vendidos.	RF-04	Must	3	2	Terminada
HU-04	Como administrador quiero listar los destinos y su disponibilidad.	RF-05	Must	1	2	Terminada
HU-05	Como administrador quiero crear un paquete con fechas, cupo y margen.	RF-06	Must	3	2	Terminada
HU-06	Como administrador quiero agregar entre 2 y 5 destinos a un paquete.	RF-07	Must	3	2	Terminada
HU-07	Como administrador quiero publicar un paquete con su precio calculado y fijo.	RF-08, 09	Must	3	2	Terminada
HU-08	Como equipo queremos una base de datos con integridad referencial para no perder información.	RF-20, RNF-09, 11	Must	5	2	Terminada
HU-09	Como visitante quiero registrarme para poder reservar.	RF-12, RNF-01, 06, 08	Must	5	3	Terminada
HU-10	Como usuario quiero iniciar sesión de forma segura según mi rol.	RF-13, 14, RNF-04, 05	Must	5	3	Terminada
HU-11	Como cliente de la agencia quiero que mi RUT y teléfono queden protegidos.	RNF-02, 03	Must	5	3	Terminada

ID	Historia de usuario	Requerimientos	Prioridad	Puntos	Sprint	Estado
HU-12	Como visitante quiero ver los paquetes vigentes con su precio y cupo.	RF-10, 11	Must	2	3	Terminada
HU-13	Como cliente quiero reservar un paquete y conocer el total.	RF-16 a 19	Must	8	3	Terminada
HU-14	Como cliente quiero consultar el historial de mis reservas.	RF-21	Must	2	3	Terminada
HU-15	Como usuario quiero cerrar sesión.	RF-15	Should	1	3	Terminada
HU-16	Como cliente quiero cancelar una reserva no pagada.	RF-22	Should	2	3	Terminada
HU-17	Como administrador quiero marcar una reserva como pagada.	RF-23	Should	2	3	Terminada
HU-18	Como equipo queremos bloquear el acceso tras 3 intentos fallidos.	RNF-05	Should	3	3	Terminada
HU-19	Como cliente quiero ser advertido si reservo dos veces el mismo paquete.	RF-24	Could	2	3	Terminada
HU-20	Como administrador quiero completar un paquete en borrador y publicarlo.	RF-28	Should	2	3	Terminada
HU-21	Como administrador quiero eliminar un paquete sin reservas.	RF-29	Should	2	3	Terminada
HU-22	Como administrador quiero ver todas las reservas sin exponer datos sensibles.	RF-27	Should	2	3	Terminada

8.3 Sprint Backlog

Sprint 1 · Semana 15 · Objetivo: entender el problema y modelar la solución

Tarea	Responsable	Horas	Entregable
Formar el equipo, definir roles y canal de comunicación	Gianni Antonietti	1	Tabla de roles
Analizar el caso y las entrevistas; listar problemas P1 a P9	Gianni Antonietti	3	Análisis de la problemática
Redactar, clasificar y priorizar RF y RNF	Gianni Antonietti	5	Sección de requerimientos
Diagrama de casos de uso y especificación de CU-06	Gianni Antonietti	3	Figura 1
Diagramas BPMN de reserva, paquete y registro	Maximiliano Montoya	4	Figuras 2 a 4
Diagrama de clases UML (con apoyo de IA, revisado críticamente)	Ambos	4	Figura 5 y registro de prompts
Matrices de trazabilidad y Product Backlog	Gianni Antonietti	3	Secciones 5, 7 y 8

Sprint 2 · Semana 16 · Objetivo: catálogo y paquetes funcionando con persistencia

Tarea	Responsable	Horas	Entregable
Crear esquema de base de datos y módulo de conexión (HU-08)	Maximiliano Montoya	4	Base de datos con tablas y claves foráneas
Implementar clases Destino y Catalogo con su CRUD (HU-01 a HU-04)	Maximiliano Montoya	6	Menú de destinos operativo
Implementar clase Paquete: creación, destinos y precio (HU-05 a HU-07)	Gianni Antonietti	6	Menú de paquetes operativo
Validaciones de entrada de destinos y paquetes	Maximiliano Montoya	3	Funciones de validación
Pruebas con datos del caso (6 destinos, 3 paquetes)	Gianni Antonietti	2	Evidencia de pruebas

Sprint 3 · Semana 17 · Objetivo: usuarios, seguridad, reservas y cierre del proyecto

Tarea	Responsable	Horas	Entregable
-------	-------------	-------	------------

Registro con hash de contraseña y validación de RUT, correo y teléfono (HU-09)	Maximiliano Montoya	5	Registro seguro
Inicio de sesión, roles y menús por rol (HU-10, HU-15)	Maximiliano Montoya	4	Autenticación
Cifrado y enmascaramiento de RUT y teléfono (HU-11)	Maximiliano Montoya	4	Datos sensibles protegidos
Listado de paquetes vigentes con cupo (HU-12)	Gianni Antonietti	2	Consulta de disponibilidad
Reserva con validación de cupo, fecha y total (HU-13)	Ambos	6	Reserva operativa
Historial de reservas del cliente (HU-14)	Gianni Antonietti	2	Consulta de historial
Cancelar reserva, ver reservas y marcar pagada (HU-16, HU-17, HU-22)	Gianni Antonietti	3	Gestión de reservas
Modificar destino, completar borrador y eliminar paquete (HU-02, HU-20, HU-21)	Maximiliano Montoya	3	CRUD completo
Pruebas de seguridad, casos límite y análisis con SonarCloud	Ambos	4	Tabla de pruebas y código corregido
Evaluación de seguridad con apoyo de IA y ajustes	Maximiliano Montoya	3	Sección de seguridad
Actualizar diagrama de clases según el código final	Gianni Antonietti	2	Diagrama coherente
Completar y revisar el informe final	Gianni Antonietti	4	Informe final
Preparar la defensa individual	Ambos (individual)	2	Guion de defensa

8.4 Cronograma y entregables

Semana	Sprint	Entregables	Hito
15	Sprint 1	Roles, análisis, requerimientos, casos de uso, BPMN, diagrama de clases, backlog	Modelo validado por el equipo
16	Sprint 2	Base de datos, CRUD de destinos y paquetes, cálculo de precio	Catálogo funcional
17	Sprint 3	Registro, login, cifrado, reservas, historial, CRUD completo, pruebas, evaluación de seguridad, informe final	Sistema completo y documentado
18	—	Subida del informe y del código al AAI	Entrega en AAI (23:00 h) y defensa

Capacidad estimada: dos integrantes con alrededor de 12 horas semanales cada uno (24 horas por sprint). Los Sprints 1 y 2 planifican entre 21 y 23 horas, con margen para imprevistos. El Sprint 3 concentró la implementación de las reservas y el cierre del proyecto, por lo que en esa semana el equipo aumentó su dedicación. La entrega se sube con al menos un día de anticipación, como recomienda la guía.

9. Supuestos

El caso no entrega toda la información. Donde falta un dato, se adoptó el siguiente supuesto, con su fundamento:

Vacío del caso	Supuesto adoptado	Fundamento
Qué ocurre si un cliente desiste	Solo se cancela una reserva pendiente y antes de la fecha de salida. La reserva pasa a cancelada (no se borra) y sus personas vuelven al cupo.	Una reserva pagada implicaría una devolución, y el pago está fuera del alcance. No borrarla conserva el historial que pide Ignacio Salas.
Cómo termina la temporada de un paquete	La vigencia se calcula comparando la fecha de salida con la fecha del día (<code>Paquete.estaVigente()</code>); no se guarda un estado «finalizado». Un paquete vencido deja de ofrecerse pero sigue en el historial.	Resuelve P3 sin perder las reservas antiguas y sin un proceso que actualice estados.
Quién modifica el catálogo	Solo el rol Administrador.	Hoy los tres socios editan sin control (P9); separar roles limita los cambios a quien corresponde.
Cómo se crean los administradores	En la primera ejecución el sistema pide crear la cuenta del administrador; el registro público solo crea clientes.	Evita contraseñas escritas en el código y que cualquier persona se registre como administrador.
Qué datos de un destino se modifican	Se puede modificar el nombre, la zona, la descripción, la duración o el costo base; el nombre nuevo tampoco puede repetirse.	El catálogo debe mantenerse al día (P6) sin romper R1. Cambiar el costo no altera los paquetes publicados (R7).
Modificación de paquetes	Solo un paquete en borrador recibe destinos y se publica; un paquete publicado no se modifica.	Su precio quedó fijado al publicar (R7); cambiarlo reproduciría el problema P4.
Eliminación de paquetes	Solo se elimina un paquete sin reservas, y previa confirmación.	Conserva el historial de los clientes (R12); la base de datos además lo impide con <code>ON DELETE RESTRICT</code> .
Datos sensibles en listados del administrador	Al ver todas las reservas, el RUT y el teléfono aparecen enmascarados.	R17 prohíbe mostrarlos en listados; el administrador no necesita el dato completo para gestionar reservas.
Cómo se registra el pago	La reserva nace pendiente y el administrador la marca pagada tras verificar la transferencia.	La pasarela y la verificación automática están fuera del alcance.
Paquete con menos de dos destinos	Queda guardado como borrador y no se ofrece hasta completarse.	R3 exige al menos dos destinos para un paquete válido.
Montos y redondeo	Los montos se manejan en pesos enteros; el precio con margen se redondea al peso.	El peso chileno no usa decimales y así se evitan errores de redondeo.
Límites de los datos	Costo hasta \$50.000.000, duración hasta 60 días, cupo hasta 100 personas y margen hasta 100 %.	Evitan errores de digitación como el del cero de más de Paulina (P5).
Base de datos	SQLite, incluida en Python.	No requiere servidor y cubre con holgura el volumen de la agencia (214 reservas por temporada).

10. Implementación en Python

La solución se implementó en Python como una aplicación de consola con persistencia en SQLite. El código se organiza en capas: el modelo de dominio, la persistencia, la seguridad, los servicios externos y la interfaz.

10.1 Estructura del proyecto

Archivo	Capa	Responsabilidad
main.py	Modelo de dominio	Las 8 clases del diagrama (Usuario, Cliente, Administrador, Sesión, Catálogo, Destino, Paquete y Reserva), con las reglas de negocio R1 a R17. Es también el punto de partida del programa.
database.py	Persistencia	Clase BaseDatos: conexión SQLite, creación de tablas y operaciones CRUD de todas las entidades.
cifrado.py	Seguridad	Clase Cifrador: cifra y descifra el RUT y el teléfono con Fernet; lee la llave desde .env.
servicios_externos.py	Integración	Clases ServicioClima (Open-Meteo) y ServicioCambio (mindicator.cl), con manejo de errores.
interface.py	Presentación	Menús de consola por rol, lectura y sanitización de entradas, y coordinación entre el modelo y la base de datos.
datos_prueba.py	Pruebas	Carga clientes, destinos, paquetes y reservas de prueba usando las mismas clases del sistema.
requirements.txt · .gitignore	Configuración	Dependencias de PyPI (requests, cryptography, python-dotenv) y exclusión de .env y bases de datos del repositorio.

10.2 Correspondencia con el diseño y principios de POO

Cada clase del diagrama de la Figura 5 existe en main.py con los mismos atributos, métodos y relaciones. Los principios de la programación orientada a objetos se aplican así:

Principio	Cómo se aplica en el código
Encapsulamiento	Todos los atributos son privados (self.__nombre, dos guiones bajos) y solo se modifican mediante métodos que validan. Las listas internas se entregan como copia (getDestinos(), listarMisReservas()) para que nadie las altere desde fuera. totalCobrado no tiene método para cambiarlo (R13) y calcularPrecio() es privado, de modo que el precio solo se calcula al publicar (R7).
Herencia	Cliente y Administrador heredan de Usuario el hash de la contraseña, su verificación y el inicio de sesión, que se escriben una sola vez.
Polimorfismo	El inicio de sesión trata a cualquier Usuario de la misma forma: llama a iniciarSesion() sin importar si es Cliente o Administrador, y según el tipo del objeto se muestra el menú que corresponde.
Abstracción	Cada clase oculta los detalles de su responsabilidad: Catálogo aplica R1 y R8 sin que la interfaz conozca las reglas, BaseDatos oculta el SQL, Cifrador oculta el algoritmo de cifrado y los servicios externos ocultan las llamadas HTTP.
Validación en dos barreras	La interfaz valida cada dato al ingresarlo, y los constructores de Destino y Paquete vuelven a validar R2, R5 y R6. Así, un dato inválido no puede crear un objeto aunque llegue por otro camino.

Respecto del diseño inicial, el código agregó métodos que el diagrama de la Figura 5 ya incluye: getId() y asignarId() para recibir el identificador que genera la base de datos; consultas como getNombre() o getFechaSalida() que la interfaz necesita para mostrar información; contieneDestino() y destinoIncluidoEnPaquete() para decidir si un destino se elimina o se marca no disponible (R8); métodos para agregar o quitar reservas que permiten reconstruir el modelo al iniciar y revertir una operación si falla la base de datos; y, en la última iteración, actualizarInformacion() para modificar un destino, existeNombreDestino() para aplicar R1 también al cambiar un nombre, listarPaquetes() y getReservas() para que el administrador gestione borradores y reservas, y

getToken() para leer el token de la sesión. Estos métodos no estaban en el diseño inicial: se incorporaron al diagrama de la Figura 5 y el archivo uml.png del proyecto se reemplazó por la versión actualizada, de modo que código y diagrama coinciden. Por último, para completar RF-24 se agregó tieneReservaActiva() en Cliente, que advierte cuando el cliente ya tiene una reserva pendiente o pagada en el mismo paquete. Usuario se mantuvo como clase concreta, tal como en el diagrama.

10.3 Persistencia de datos

Se usa sqlite3, incluida en la biblioteca estándar de Python, porque no requiere instalar ni administrar un servidor y cubre el volumen de la agencia. La base de datos se crea automáticamente en la primera ejecución con cinco tablas relacionadas:

Tabla	Contenido	Restricciones de integridad
usuarios	Clientes y administradores, con su rol	Correo único sin distinguir mayúsculas. Contraseña guardada como hash. RUT y teléfono cifrados.
destinos	Catálogo de destinos	Nombre único sin distinguir mayúsculas (R1). Disponibilidad y fecha de actualización.
paquetes	Paquetes con su precio y estado	Precio por persona guardado al publicar (R7).
paquete_destinos	Destinos de cada paquete, en orden	Clave primaria compuesta (no repite destino, R3). Borrar un paquete borra sus vínculos (CASCADE); un destino usado no se puede borrar (RESTRICT, R8).
reservas	Reservas con su total y estado	Claves foráneas a usuarios y paquetes con RESTRICT: no se pierde el historial (R12).

Las claves foráneas se activan con PRAGMA foreign_keys = ON. Cada escritura se ejecuta dentro de una transacción (bloque with de la conexión) y todas las consultas usan parámetros (?). Al iniciar, la función cargar_datos() reconstruye los objetos y sus relaciones desde la base de datos, conservando los identificadores, los estados y los precios fijados. Si una escritura falla, la interfaz revierte también el cambio en memoria (por ejemplo, quita la reserva del paquete y del cliente).

10.4 Operaciones CRUD

Cada entidad del negocio tiene sus cuatro operaciones disponibles desde el menú, y la capa de datos (BaseDatos) las implementa con SQL parametrizado:

Entidad	Crear	Consultar	Modificar	Eliminar
Destino	Registrar destino	Listar destinos	Modificar destino (nombre, zona, descripción, duración o costo)	Retirar destino: se elimina, o se marca no disponible si está en un paquete (R8)
Paquete	Crear paquete	Ver paquetes vigentes	Completar paquete en borrador y publicarlo	Eliminar paquete sin reservas, con confirmación
Reserva	Reservar	Mis reservas (cliente) · Ver reservas (administrador)	Marcar reserva como pagada	Cancelar una reserva: cambia su estado y conserva el historial
Usuario	Registrarse como cliente · administrador inicial	Inicio de sesión y carga al iniciar	actualizar_usuario() en la capa de datos	eliminar_usuario() en la capa de datos

El menú del administrador tiene 11 opciones (listar, registrar, modificar y retirar destinos; crear, completar y eliminar paquetes; ver paquetes vigentes; ver reservas; marcar reserva como pagada; y ver el clima) y el del cliente, 7 (ver paquetes, reservar, mis reservas, cancelar una reserva, ver el clima, ver el precio en otra moneda y cerrar sesión). Las acciones que no se pueden deshacer piden confirmación (s/n). Las operaciones sobre usuarios se dejaron solo en la capa de datos, porque el caso no pide modificar ni eliminar clientes desde el sistema.

10.5 Evidencia de ejecución

Extracto de una ejecución real del programa con una base de datos nueva, siguiendo el ejemplo del caso (paquete «Norte Grande» con Salar de Surire y Valle del Elqui, margen 20 %):

```
=== Viajes Aventura ===
Primera ejecución: cree la cuenta del administrador.
Inicio de sesión
  Bienvenido/a, Gianni Antonietti.
Nuevo destino -> Destino registrado.          (Salar de Surire, $310.000)
Nuevo destino -> Destino registrado.          (Valle del Elqui, $120.000)
Nuevo destino -> Ya existe un destino con ese nombre. ("valle del elqui", R1)
Nuevo paquete
  Destino «Salar de Surire» agregado (1 de 5).
  Destino «Valle del Elqui» agregado (2 de 5).
  Paquete publicado a $516.000 por persona.

Registro de cliente
  RUT no válido. Escríbalo sin puntos y con guion, ej. 19616711-0
  Teléfono no válido. Escríbalo sin espacios, ej. +56912345678
  La contraseña debe combinar letras y números.
  Cliente registrado.
Inicio de sesión
  Correo o contraseña incorrectos.
Inicio de sesión
  Bienvenido/a, Carolina Reyes.
Reservar paquete (13 personas)
  No se pudo reservar: No hay cupo suficiente en el paquete.
Reservar paquete (2 personas)
  Reserva creada. Total: $1.032.000 (estado: pendiente).
Reservar paquete (3 personas)
  Reserva creada. Total: $1.548.000 (estado: pendiente).
Cancelar una reserva
  ¿Confirma la cancelación? (s/n): s
  Reserva cancelada. Se liberaron 2 cupo(s).
Mis reservas
  - Norte Grande en 5 días · 2 persona(s) · $1.032.000 · cancelada
  - Norte Grande en 5 días · 3 persona(s) · $1.548.000 · pendiente

(administrador) Ver reservas
  - Carolina Reyes (RUT *****8-5, tel. *****678) · Norte Grande en 5 días · 3 persona(s) ·
$1.548.000 · pendiente
(administrador) Marcar reserva como pagada
  Reserva marcada como pagada.
(administrador) Eliminar paquete
  No se puede eliminar: el paquete tiene reservas registradas.
```

Contenido de la tabla usuarios después de la ejecución (valores recortados): la contraseña está guardada como iteraciones\$sal\$hash y el RUT y el teléfono como texto cifrado.

```
contrasena_hash = '600000$c70cdb7eb2649ea0158a099...'
rut              = 'gAAAAABqwDVB3S9VBqu1...'
```

11. Seguridad del sistema

11.1 Librerías utilizadas

Librería	Origen	Uso en el sistema
hashlib, hmac, secrets	Biblioteca estándar de Python	Hash PBKDF2-HMAC-SHA256 de contraseñas, comparación en tiempo constante y generación del token de sesión.
cryptography (Fernet)	PyPI · Python Cryptographic Authority	Cifrado autenticado del RUT y el teléfono.
python-dotenv	PyPI	Lectura de la llave de cifrado desde el archivo .env.
sqlite3	Biblioteca estándar de Python	Base de datos con consultas parametrizadas.
requests	PyPI	Consumo de servicios externos con tiempo máximo de espera.

Las dependencias externas están declaradas en requirements.txt con versión mínima, y se instalan con pip desde el repositorio oficial PyPI.

11.2 Autenticación

- **Hash de contraseñas (R10):** cada contraseña se procesa con PBKDF2-HMAC-SHA256, una sal aleatoria de 16 bytes (os.urandom) y 600.000 iteraciones, la cifra que recomienda OWASP para este algoritmo. Se guarda como iteraciones\$sal\$hash: al guardar el número de iteraciones junto al hash, las cuentas creadas antes con 200.000 se siguen verificando sin obligar a cambiar la contraseña. La contraseña original nunca se almacena.
- **Verificación en tiempo constante:** la comparación usa hmac.compare_digest, que no revela información por el tiempo que tarda en responder.
- **Ingreso oculto:** la contraseña se muestra como asteriscos y, al crear una cuenta, se pide dos veces.
- **Política de contraseña:** entre 8 y 64 caracteres, combinando letras y números. El máximo evita que una contraseña enorme vuelva lento el cálculo del hash.
- **Sesiones:** al iniciar sesión se crea un objeto Sesion con un token aleatorio (secrets.token_hex) que expira a los 30 minutos y se cierra al salir del menú.

11.3 Validación de credenciales y control de acceso

- **Mensaje genérico:** si el correo no existe o la contraseña es incorrecta, el mensaje es el mismo («Correo o contraseña incorrectos»), para no revelar qué correos están registrados.
- **Bloqueo por intentos:** tras 3 intentos fallidos con un mismo correo, ese correo no puede iniciar sesión durante 1 minuto. Se cuenta aunque el correo no exista.
- **Control por rol:** el menú se elige según el tipo de usuario. Un cliente no tiene acceso a las opciones de administración.
- **Sesión vigente y propia (R11):** reservar, consultar reservas y usar servicios externos verifican que la sesión no esté cerrada ni vencida y que pertenezca al usuario. listarMisReservas() lanza un error de permiso si la sesión es de otro usuario, así cada cliente ve solo sus reservas.

11.4 Protección de la información sensible

- **Cifrado del RUT y el teléfono (R17):** se cifran con Fernet antes de guardarlos. Fernet combina AES-128 en modo CBC con una firma HMAC-SHA256, por lo que protege la confidencialidad y además detecta si el dato fue alterado (integridad).
- **Llave fuera del código:** la llave se lee del archivo .env (variable VIAJES_CLAVE_CIFRADO), que está en .gitignore. Si no existe, se genera la primera vez.

Si la llave no corresponde a los datos, se muestra un mensaje genérico sin la llave ni el detalle del error.

- **Migración de datos antiguos:** al abrir el programa, los clientes guardados antes de existir el cifrado se cifran automáticamente.
- **Enmascaramiento:** la clase Cliente ofrece el RUT y el teléfono solo enmascarados (se muestran los 3 últimos caracteres). No se muestran completos en listados (ni siquiera en el listado de reservas del administrador) ni en mensajes de error.

11.5 Validación de entradas y otros controles

- **Sanitización:** todo texto ingresado se limpia de caracteres de control y secuencias ANSI, se normalizan acentos y espacios, y se aplica un largo máximo.
- **Formatos estrictos:** el RUT solo se acepta como 12345678-9 y con su dígito verificador correcto (módulo 11), el teléfono como +56912345678 o 912345678, y los nombres de clientes, administradores y destinos no admiten dígitos. Los nombres de paquetes sí, porque son nombres comerciales (como «Norte Grande en 5 días», del caso). Así lo aceptado coincide con lo que indica cada mensaje. Los números tienen mínimos y máximos.
- **Expresión regular segura:** la validación del correo usa una expresión regular sin retroceso excesivo (backtracking), que responde al instante incluso con entradas maliciosas largas y evita un ataque de denegación de servicio por expresiones regulares (ReDoS).
- **Confirmación de acciones irreversibles:** cancelar una reserva y eliminar un paquete piden confirmación (s/n).
- **Inyección SQL:** todas las consultas usan parámetros (?), nunca texto concatenado.
- **Errores controlados:** los fallos de la base de datos, del almacenamiento, del cifrado y de los servicios externos se muestran con mensajes genéricos, sin rutas ni detalles internos.
- **Servicios externos:** la ciudad que se envía a la API solo admite letras, espacios, guiones y apóstrofes; las consultas tienen un tiempo máximo de 10 segundos y se validan los datos recibidos antes de mostrarlos.

11.6 Pruebas de seguridad

#	Prueba	Resultado esperado	Resultado obtenido
1	Iniciar sesión con contraseña incorrecta	Mensaje genérico	«Correo o contraseña incorrectos.» ✓
2	Cuarto intento tras 3 fallos con el mismo correo	Acceso bloqueado	«Demasiados intentos fallidos. Espere 60 segundos» ✓
3	Correo con inyección: admin' OR 1=1 --@x.cl	Rechazado	«Ingrese un correo con formato válido.» ✓
4	RUT sin guion (123456789) o con dígito verificador incorrecto	Rechazado	«RUT no válido.» ✓
5	Contraseña «aaaaaaaa» (sin números)	Rechazada	«La contraseña debe combinar letras y números.» ✓
6	Revisar la tabla usuarios con un visor	Sin datos legibles	Hash 600000\$sal\$hash; RUT y teléfono cifrados (gAAAAA...) ✓
7	Iniciar sesión con una cuenta creada con 200.000 iteraciones	Sigue funcionando	La contraseña antigua se verifica correctamente ✓
8	Abrir el programa con una llave equivocada	Mensaje genérico, sin la llave	Mensaje de datos protegidos ilegibles ✓
9	Ciudad «<script>» o «Vicuña;DROP TABLE» para el clima	Rechazada	Se pide solo el nombre de la ciudad ✓
10	Reservar con una sesión vencida	Volver al menú principal	«Su sesión expiró.» ✓
11	Reservar más personas que el cupo disponible	Rechazada	«No hay cupo suficiente en el paquete.» ✓

#	Prueba	Resultado esperado	Resultado obtenido
12	Ver reservas como administrador	RUT y teléfono enmascarados	«RUT *****8-5, tel. *****678»
13	Eliminar un paquete que tiene reservas	Rechazado	«No se puede eliminar: el paquete tiene reservas registradas.» ✓
14	Nombre de cliente con números («Ana2»)	Rechazado	«El nombre no puede contener números.» ✓
15	Cerrar la entrada del programa (Ctrl+Z o Ctrl+D)	Salida controlada, sin rutas internas	«Programa finalizado.» ✓
16	Reservar dos veces el mismo paquete, y de nuevo tras cancelar	Advertencia solo con una reserva activa	Con reserva activa: «Ya tiene una reserva activa en este paquete.» y se pide s/n; tras cancelar, sin advertencia ✓

Las pruebas 1 a 6 y 11 a 16 se ejecutaron sobre la versión final con una base de datos nueva; las pruebas 7 a 10 están documentadas en el registro de validación del equipo (validacion_ia.md).

11.7 Evaluación de la seguridad con apoyo de IA

La seguridad se evaluó de dos formas: con una herramienta de IA, a la que se pidió revisar el código contra la rúbrica y las recomendaciones de OWASP, y con el análisis estático de SonarCloud, que no reportó vulnerabilidades ni puntos críticos de seguridad (security hotspots). Cada hallazgo se evaluó críticamente: casi todos se aplicaron y dos se aceptaron como riesgo justificado para una aplicación de consola local.

#	Hallazgo	Riesgo	Recomendación	Decisión
S1	Las bases de respaldo (viajes_aventura_respaldo*.db) guardan RUT y teléfono sin cifrar	Alto	Eliminarlas de la carpeta y de la entrega	Aplicada: se eliminaron y la entrega solo incluye la base de datos cifrada
S2	El archivo .env con la llave de cifrado puede quedar dentro del zip de entrega	Alto	Entregar sin .env; el programa genera una llave nueva al iniciar	Aplicada: el .env no se incluye en la entrega
S3	PBKDF2 con 200.000 iteraciones, menos que las 600.000 que recomienda OWASP para SHA-256	Medio	Subir las iteraciones y guardarlas dentro del hash para no invalidar contraseñas existentes	Aplicada: 600.000 iteraciones, guardadas en el hash; las cuentas antiguas se siguen verificando
S4	Cliente tiene getRut() y getTelefono() públicos sin uso, que entregan el dato completo	Medio	Eliminarlos: el sistema solo necesita los métodos enmascarados	Aplicada: se eliminaron; Cliente solo expone el RUT y el teléfono enmascarados
S5	El registro responde «Ese correo ya está registrado», lo que permite saber qué correos existen	Bajo	Aceptado: la persona necesita saber por qué no puede registrarse; el login sí es genérico	Aceptado como riesgo justificado
S6	Los intentos fallidos se guardan en memoria y se reinician al cerrar el programa	Bajo	Persistirlos si el sistema pasa a ser multiusuario o en red	Aceptado: aplicación de consola local
S7	Cerrar la entrada (Ctrl+Z o Ctrl+D) o interrumpir con Ctrl+C muestra un error con rutas internas	Bajo	Capturar EOFError y KeyboardInterrupt en iniciar() con un mensaje genérico	Aplicada: el programa termina con «Programa finalizado.»
S8	quitarReservaNoPersistida() repite el comportamiento de quitarReservaPersistida()	Bajo (calidad)	Unificar en un solo método	Aplicada: se eliminó el método duplicado
S9	La expresión regular del correo podía retroceder en exceso con entradas maliciosas (ReDoS), señalado por SonarCloud	Medio	Usar una expresión sin retroceso	Aplicada: responde al instante con entradas maliciosas largas
S10	Se aceptaban contraseñas débiles como «aaaaaaa» o «12345678»	Medio	Exigir letras y números	Aplicada
S11	El RUT y el teléfono se aceptaban en formatos distintos de los que indicaba el mensaje	Bajo	Exigir exactamente el formato indicado	Aplicada

#	Hallazgo	Riesgo	Recomendación	Decisión
S12	La validación de nombres sin dígitos se aplicaba también a los paquetes, e impedía registrar «Norte Grande en 5 días» del caso	Bajo (funcional)	Permitir dígitos en los nombres de paquetes	Aplicada. Por decisión del equipo se mantiene en los destinos: son lugares reales y ninguno del caso lleva números

12. Uso crítico de herramientas de IA

12.1 Herramientas utilizadas

Herramienta	Uso
Claude (claude.ai)	Propuestas del diagrama de clases, revisión del modelo contra la notación del curso, revisión de seguridad del código final y apoyo en la redacción del informe.
Claude Code (extensión de VS Code)	Implementación del modelo UML en Python, aplicación de principios POO, sanitización de entradas, navegación de menús, servicios externos y mecanismos de seguridad.
GitHub Copilot (VS Code)	Propuesta de la capa SQLite y del manejo de errores y validaciones.

En todos los casos la IA propuso y el equipo decidió: cada propuesta se revisó contra el caso, las reglas R1 a R17 y la notación vista en clases antes de adoptarla, modificarla o descartarla.

12.2 Iteraciones de propuesta del modelo

Iteración	Prompt y contexto entregado	Resultado	Decisión
1	«analiza este trabajo y creame un UML para este trabajo», adjuntando el PDF del caso.	13 clases: las 8 del dominio más servicios (ServicioAutenticacion, ServicioReservas, Validador, HashContrasena) y enumeraciones. Tipos de Java (String, boolean, LocalDate) y dependencias punteadas.	Descartada: usa relaciones que no son las cuatro del curso, tipos de otro lenguaje y clases de servicio que concentran responsabilidades.
2	«necesito que se cumpla en el diagrama, hazlo de nuevo», adjuntando las láminas de visibilidad y de las cuatro relaciones.	Tipos de Python y solo las cuatro relaciones, pero conservaba las clases de servicio.	Descartada: seguía teniendo clases tipo Gestor.
3	«hace todo lo que te mande...», adjuntando la presentación completa «Del código al diagrama».	8 clases con caja de tres pisos, firmas completas, cuatro relaciones justificadas y multiplicidad en ambos extremos.	Elegida como base y ajustada por el equipo (12.3).

Aprendizaje: los primeros prompts eran breves y el resultado mejoró a medida que se entregó contexto (el caso) y formato (las láminas de notación). En adelante conviene redactar desde el inicio con el esquema CLARO: contexto, labor, alcance, rol y orden.

12.3 Análisis del modelo propuesto por la IA

#	Aspecto	Tipo	Qué propuso la IA	Qué decidimos y por qué
1	Clases	Error	Clases de servicio (ServicioAutenticacion, ServicioReservas) que concentraban la lógica	Se eliminaron: es el error frecuente de una clase «Gestor». La autenticación quedó en Usuario y la reserva en Paquete.
2	Relaciones	Error	Dependencias punteadas entre servicios	Se reemplazaron: el curso usa solo asociación, agregación, composición y generalización.
3	Atributos	Diferencia	Tipos de Java (String, boolean, LocalDate)	Se cambiaron a str, bool y date, porque el sistema se implementa en Python.
4	Atributos	Diferencia	Montos como int, mientras la lámina del curso usa float para precios	Se mantuvo int: el peso chileno no tiene decimales y float produce errores de redondeo en dinero.
5	Clases	Diferencia	Usuario como clase abstracta	Se dejó como clase concreta, según el diagrama oficial del equipo.
6	Clases	Similitud	Identificó Cliente, Administrador, Destino, Paquete, Reserva y Catalogo	Se conservaron: coinciden con los sustantivos del análisis propio (sección 3).
7	Relaciones	Similitud	Paquete–Destino como agregación con multiplicidad 2..5	Se conservó: el destino sobrevive al paquete y R3 fija entre 2 y 5.

12.4 Análisis del código generado con IA

Decisión	Propuesta de la IA	Fundamento
Adoptado	Hash PBKDF2-SHA256 con sal y comparación con hmac.compare_digest	Cumple R10 y evita ataques por tiempo de respuesta.
Adoptado	Consultas SQL parametrizadas	Previenen la inyección SQL.
Adoptado	Mensajes de error genéricos ante fallas de SQLite	No exponen rutas ni detalles internos.
Modificado	Una función distinta para leer cada tipo de dato	Se unificó en leer_linea(): una sola función con la tecla Esc para volver.
Modificado	Un único mensaje para «destino repetido o ya hay 5»	Se separó en dos mensajes para que el usuario sepa qué regla (R3) no cumplió.
Descartado	Volver al menú con «1» o con «0»	«1» es un dato válido (1 persona, 1 día) y «0» impedía un margen de 0 %, que R6 permite. Se usó Esc.
Descartado	Guardar coordenadas en Destino para el clima	Obligaba a cambiar el diagrama y la base de datos; se pide la ciudad más cercana.
Descartado	OpenWeatherMap con llave de acceso	Se prefirieron APIs sin llave para no administrar otro secreto.
Corregido	Las validaciones de R2, R5 y R6 estaban solo en el menú	Se agregaron también en los constructores de Destino y Paquete, como segunda barrera.
Corregido	«Reservar» no verificaba que la sesión estuviera vigente	Ahora lo verifica, igual que «Mis reservas» (R11).
Corregido	Una respuesta JSON con estructura inesperada podía cerrar el programa	Se valida el formato y se convierte en un error controlado.
Corregido	Los métodos cancelar() y marcarReservaPagada() existían en las clases, pero el menú no los usaba	Se agregaron las opciones de menú, guardando primero el estado en la base de datos y después en el objeto, para que ambos no queden distintos si falla SQLite.
Adoptado	Guardar el número de iteraciones dentro del hash al subir PBKDF2 a 600.000	Permite endurecer la seguridad sin invalidar las cuentas existentes.
Descartado	Renombrar métodos y atributos a snake_case, como sugería SonarCloud	Los nombres siguen en camelCase el diagrama UML oficial; esas reglas se excluyeron del análisis.

Cada corrección se probó antes y después del cambio. El registro completo de prompts, resultados y pruebas está en el archivo validacion_ia.md del proyecto.

12.5 Modelo ajustado

El modelo final (Figuras 5 y 6) es el resultado de la tercera iteración, ajustado con las decisiones de las tablas anteriores y alineado con el código. Las diferencias respecto de la propuesta original son: sin clases de servicio en el dominio, tipos de Python, solo las cuatro relaciones del curso, montos enteros y los métodos de consulta que la implementación necesitó.

13. Conclusiones

El sistema desarrollado reemplaza la planilla, el cuaderno y los mensajes que usaba Viajes Aventura por una aplicación orientada a objetos con persistencia y control de acceso. Cada problema detectado en el caso tiene una respuesta concreta en el sistema: el cupo y la fecha se validan antes de reservar, el precio queda fijo al publicar, el catálogo no admite nombres repetidos ni pierde destinos usados, y cada cliente consulta su propio historial.

La seguridad fue el eje del desarrollo: las contraseñas se guardan como hash, el RUT y el teléfono se cifran con una llave que no está en el código, el acceso se controla por sesión y por rol, y toda entrada se valida antes de usarse. La revisión final con apoyo de IA identificó mejoras adicionales, priorizadas por su riesgo.

El trabajo confirmó el método visto en clases: primero se entiende el problema, después se modela y al final se codifica. La IA aceleró cada etapa, pero sus propuestas requirieron revisión crítica: varias se corrigieron o descartaron por no ajustarse al caso, a la notación del curso o a buenas prácticas de seguridad.

Referencias

Cuevas Álvarez, A. (2016). Python 3. RA-MA Editorial.

International Organization for Standardization. (2023). ISO/IEC 25010:2023 Systems and software engineering — Product quality model. ISO.

Jiménez de Parga, C. (2021). UML: arquitectura de aplicaciones en Java, C++ y Python (1.^a ed.). RA-MA.

Object Management Group. (2014). Business Process Model and Notation (BPMN), versión 2.0.2. OMG.

Object Management Group. (2017). Unified Modeling Language (UML), versión 2.5.1. OMG.

OWASP Foundation. (s. f.). Password Storage Cheat Sheet. OWASP Cheat Sheet Series.

Python Cryptographic Authority. (s. f.). Fernet (symmetric encryption). Documentación de cryptography.

Python Software Foundation. (s. f.). hashlib, hmac, secrets y sqlite3. Documentación de la biblioteca estándar de Python 3.

Sánchez Palacio, A. (2025). ChatGPT y OpenAI: desarrollo y uso de herramientas de inteligencia artificial generativa. RA-MA Editorial.

Schwaber, K. y Sutherland, J. (2020). La Guía de Scrum. Scrum.org.